

# TD5 : Interrogations avancées en SQL — Corrigé enseignant

## 5de765a

Rosine Cicchetti - Lotfi Lakhal - Mickaël Martin Nevot



Cette œuvre de Rosine Cicchetti est mise à disposition sous licence Creative Commons Attribution - Utilisation non commerciale - Partage dans les mêmes conditions.

Document en ligne : [www.mickael-martin-nevot.com](http://www.mickael-martin-nevot.com)

---

## 1 Rappel : Prise en main d'un éditeur Oracle

Utilisez une des deux solutions ci-dessous.

### 1.1 Oracle Live SQL

Vous pouvez utiliser directement, sans installation ou configuration, l'interpréteur en ligne : <https://livesql.oracle.com>.

### 1.2 Oracle Cloud Free Tier

Mettez en place une solution d'hébergement en ligne d'un SGBD Oracle. Vous pouvez pour cela consulter le document [Vade-Mecum mise en place d'un hébergement Oracle Cloud Free Tier](#).

Ajouter ensuite une base de données nommée `zenetude-bd`.

## 2 Rappel : Généralités

Voici la convention de nommage proposé dans cet enseignement :

- **mots clefs** : en lettre capitales (*upper case*) ;
- **relation** : première lettre de chaque mot en capitale (*pascal case*) ;
- **attributs** : premier mot en minuscule et première lettre de chaque mot suivant en capitale (*camel case*) ;
- **domaine** : en lettre capitales (*upper case*) au format `D_XXX`<sup>1</sup>.

Pour rappel, les valeurs saisies dans une base de données, comme les chaînes de caractères, sont sensibles à la casse et, par convention, sont saisies **en lettres capitales** (*upper case*), sans diacritique, dans le cadre de cet enseignement.

---

1. Les domaines identiques sont surlignés avec la même couleur.

### 3 Rappel : Base de données exemple

La base de données exemple, Gestion pédagogique, utilisée dans les documents de travaux dirigés de cet enseignement, permet de gérer une formation en informatique, avec ses professeurs, ses étudiants, ses modules ainsi que leurs enseignements et notations.

#### 3.1 Dictionnaire de données

La base de données Gestion pédagogique a été élaborée à partir du dictionnaire des données suivant<sup>2</sup> :

**Table 1** – Dictionnaire des données de la base de données exemple

| Attribut    | Description                                           | Domaine                     | Remarques       |
|-------------|-------------------------------------------------------|-----------------------------|-----------------|
| numEt       | Numéro d'un étudiant                                  | D_NUMET :<br>NUMBER(6,0)    | Valeurs uniques |
| nomEt       | Nom d'un étudiant                                     | D_NOM :<br>VARCHAR2(30)     |                 |
| prenomEt    | Prénom d'un étudiant                                  | D_PRENOM :<br>VARCHAR2(20)  |                 |
| cpEt        | Code postal d'un étudiant                             | D_CP : VARCHAR2(5)          |                 |
| villeEt     | Ville d'un étudiant                                   | D_VILLE :<br>VARCHAR2(20)   |                 |
| anneeEt     | Année d'étude d'un étudiant                           | D_ANNEE :<br>NUMBER(2,0)    | [1..2]          |
| groupeEt    | Numéro de groupe d'un étudiant                        | D_GROUPE :<br>NUMBER(1,0)   | [1..5]          |
| numProf     | Numéro d'un professeur                                | D_NUMPROF :<br>NUMBER(3,0)  | Valeurs uniques |
| nomProf     | Nom d'un professeur                                   | D_NOM :<br>VARCHAR2(20)     |                 |
| prenomProf  | Prénom d'un professeur                                | D_PRENOM :<br>VARCHAR2(20)  |                 |
| adresseProf | Adresse d'un professeur                               | D_ADRESSE :<br>VARCHAR2(40) |                 |
| cpProf      | Code postal d'un professeur                           | D_CP : VARCHAR2(5)          |                 |
| villeProf   | Ville d'un professeur                                 | D_VILLE :<br>VARCHAR2(20)   |                 |
| specProf    | Code d'une matière dont un professeur est spécialiste | D_CODE :<br>VARCHAR2(7)     |                 |
| code        | Code d'un module                                      | D_CODE :<br>VARCHAR2(7)     | Valeurs uniques |
| libelle     | Libellé d'un module                                   | D_LIBELLE :<br>VARCHAR2(30) |                 |
| heureCMPrev | Nombre d'heures de CM prévues pour un module          | D_NBHEURE :<br>NUMBER(3,0)  |                 |

2. Les types syntaxiques, utilisés pour la description des domaines, sont disponibles dans Oracle.

| Attribut    | Description                                              | Domaine                        | Remarques                         |
|-------------|----------------------------------------------------------|--------------------------------|-----------------------------------|
| heureCMReal | Nombre d'heures de CM réalisées pour un module           | D_NBHEURE :<br>NUMBER(3,0)     |                                   |
| heureTPPrev | Nombre d'heures de TP prévues pour un module             | D_NBHEURE :<br>NUMBER(3,0)     |                                   |
| heureTPReal | Nombre d'heures de TP réalisées pour un module           | D_NBHEURE :<br>NUMBER(3,0)     |                                   |
| discipline  | Discipline enseignée                                     | D_DISCIPLINE :<br>VARCHAR2(15) | {INFORMATIQUE,<br>MATHS, GESTION} |
| coefCC      | Coefficient du contrôle continu pour un module           | D_COEF :<br>NUMBER(3,0)        | [0..100]                          |
| coefTest    | Coefficient du test pour un module                       | D_COEF :<br>NUMBER(3,0)        | [0..100]                          |
| resp        | Numéro d'un professeur responsable d'une matière         | D_NUMPROF :<br>NUMBER(3,0)     |                                   |
| codePere    | Code du module père d'un module                          | D_CODE :<br>VARCHAR2(7)        |                                   |
| moyCC       | Moyenne de contrôle continu d'un étudiant dans un module | D_NOTE :<br>NUMBER(2,0)        | [0..20]                           |
| moyTest     | Moyenne de test d'un étudiant dans un module             | D_NOTE :<br>NUMBER(2,0)        | [0..20]                           |

#### Remarques

Les modules forment une hiérarchie entre eux. Quel que soit leur niveau dans la hiérarchie, tous les modules possèdent les mêmes attributs. La racine de l'arbre ainsi formé par cette hiérarchie est le programme pédagogique national de formation en informatique. Il se décompose en véritables modules, eux-mêmes se décomposant en sous-modules et ainsi de suite jusqu'aux feuilles qui correspondent aux matières. Les matières sont les seuls modules pouvant être enseignés et susceptibles de recevoir des notes. Certains modules sont associés à une discipline.

La hiérarchie des modules de l'extension de la base de données exemple est représentée sous forme d'arborescence des codes en figure 1.

Les coefficients de contrôle continu et de test d'un module sont des pourcentages. Ils peuvent ne pas être renseignés. Si un des deux coefficients n'est pas renseigné, l'autre ne doit pas l'être non plus. Leur somme pour un même module est donc soit de 0, soit de 100.

Nous considérons qu'il n'y a pas deux personnes homonymes ayant le même prénom et le même nom.

## 3.2 MCD

Le MCD (voir figure 2) modélise trois entités : **Module** (les unités d'enseignement organisées en hiérarchie), **Étudiant** et **Prof**. L'association **Notation** relie un étudiant à un module et porte ses moyennes de contrôle continu et de test. L'association ternaire **Enseignement** lie un



Figure 1 – Hiérarchie des modules

professeur, un module et un étudiant. Un professeur peut être **Spécialiste** d'un module (sa discipline de référence) et **Responsable** d'un module. Enfin, l'association réflexive **A pour père** matérialise la hiérarchie entre modules (voir figure 1).

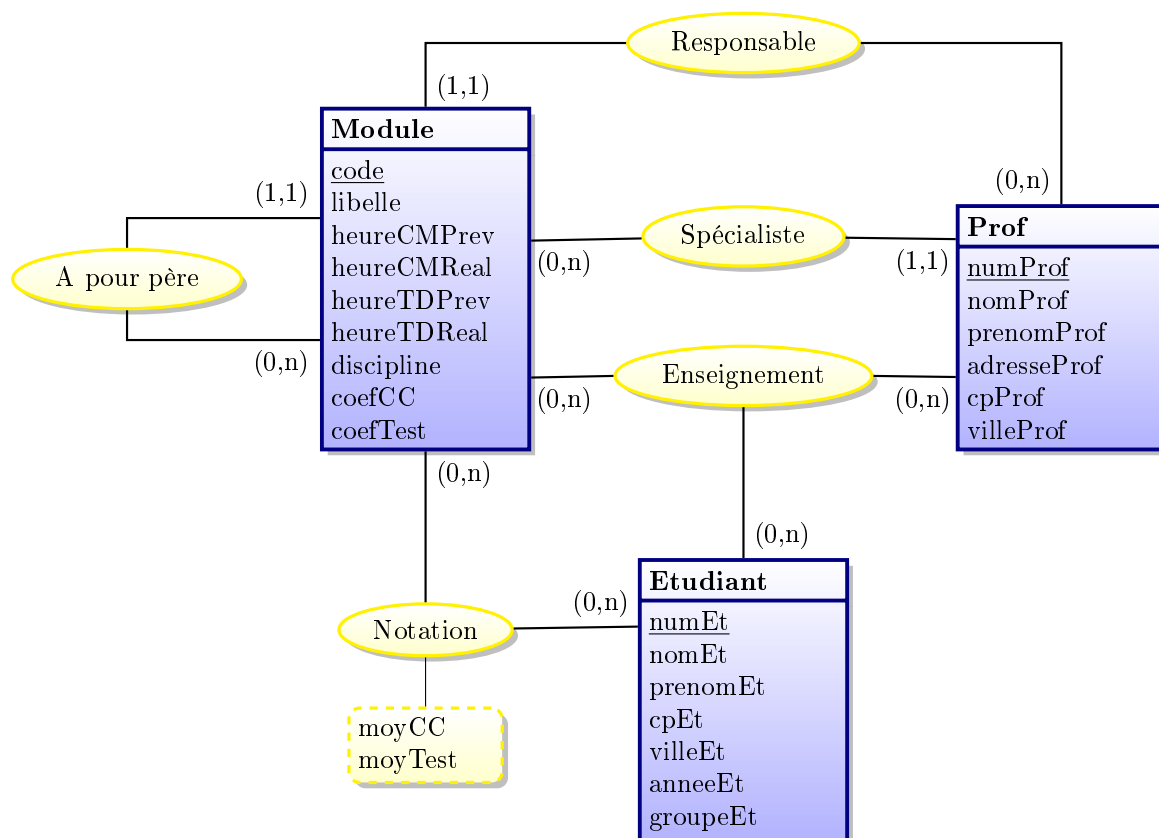


Figure 2 – Modèle conceptuel des données (MCD)

### 3.3 Schéma relationnel

Les clefs primaires sont soulignées et les clefs étrangères en *italique suivies d'un #*.

Le schéma relationnel de la base de données exemple est présenté ci-après :

**Etudiant** (numEt, nomEt, prenomEt, cpEt, villeEt, anneeEt, groupeEt)

**Professeur** (numProf, nomProf, prenomProf, adresseProf, cpProf, villeProf, *specProf#*)

**Module** (code, libelle, heureCMPprev, heureCMReal, heureTDPrev, heureTDReal, discipline, co-

efCC, coefTest, resp#, codePere#)

Note (numEt#, code#, moyCC, moyTest)

Enseigne (numEt#, code#, numProf#)

#### Remarques

La clef étrangère **resp** dans **Module** représente le numéro d'un professeur responsable d'un module<sup>a</sup> et fait référence à la clef primaire **numProf**.

La clef étrangère **specProf** dans **Professeur** représente le code d'une matière dont un professeur est spécialiste et fait référence à la clef primaire **code**.

La clef étrangère **codePere** dans la relation **Module** fait référence à la clef primaire **code**.

<sup>a</sup>. Idéalement, il ne devrait être possible d'y avoir qu'un responsable d'une matière, pas de n'importe quel module. Par souci de simplification, cette contrainte n'a pas été prise en compte dans l'intension de la base de données exemple.

Les autres clefs étrangères font référence aux clefs primaires de même nom.

## 4 Requêtes avec SQL

Formulez, en SQL, sur la base de données exemple, les requêtes d'interrogation suivantes.

### 4.1 Expression des partitionnements

**Q1** Quel est, pour chacun des numéros et noms des professeurs, et pour chacune des années, le nombre d'étudiants auxquels il a enseigné? *4 attributs, 12 tuples*

```
SELECT P.numProf, nomProf, anneeEt, COUNT(DISTINCT Et.numEt) AS nbEtudiants
FROM Enseigne E
     INNER JOIN Professeur P ON E.numProf = P.numProf
     INNER JOIN Etudiant Et ON E.numEt = Et.numEt
GROUP BY P.numProf, nomProf, anneeEt;
```

**Q2** Quel est, pour chacun des numéros et noms des professeurs responsable d'un module, le nombre de modules dans lesquels il fait cours? Formulez les requêtes suivantes en utilisant, si possible, les trois formes de jointure. *3 attributs, 5 tuples*

**Version algébrique.**

```
SELECT P.numProf, nomProf, COUNT(DISTINCT E.code) AS nbModules
FROM Enseigne E
     INNER JOIN Professeur P ON E.numProf = P.numProf
     INNER JOIN MODULE M ON P.numProf = M.resp
GROUP BY P.numProf, nomProf;
```

**Version imbriquée.**

```
SELECT P.numProf, nomProf, COUNT(DISTINCT code) AS nbModules
FROM Enseigne E
     INNER JOIN Professeur P ON E.numProf = P.numProf
WHERE
```

```

P.numProf IN (
    SELECT resp
    FROM MODULE
)
GROUP BY P.numProf, nomProf;

```

Version prédictive.

```

SELECT P.numProf, nomProf, COUNT(DISTINCT E.code) AS nbModules
FROM Enseigne E, Professeur P, MODULE M
WHERE
    E.numProf = P.numProf
    AND P.numProf = M.resp
GROUP BY P.numProf, nomProf;

```

**Q3** Donnez la moyenne des moyennes de test par groupe d'étudiants de seconde année pour le module de libellé Conception de SI. *2 attributs, 5 tuples*

```

SELECT groupeEt, AVG(moyTest) AS moyMoyTest
FROM Note N
    INNER JOIN Etudiant Et ON N.numEt = Et.numEt
    INNER JOIN MODULE M ON N.code = M.code
WHERE
    libelle = 'CONCEPTION DE SI'
    AND anneeEt = 2
GROUP BY groupeEt;

```

## 4.2 Calculs complexes

**Q4** Quels sont les numéros, noms et prénoms des enseignants ayant autant de modules que Bernard Faure? *3 attributs, 4 tuples*

V1.

```

WITH
    T (numProf, nomProf, prenomProf, nbCode) AS (
        SELECT E.numProf, nomProf, prenomProf, COUNT(DISTINCT code)
        FROM Enseigne E
            INNER JOIN Professeur P ON E.numProf = P.numProf
        GROUP BY E.numProf, nomProf, prenomProf
    )
SELECT DISTINCT numProf, nomProf, prenomProf
FROM T
WHERE
    (nomProf <> 'FAURE' OR prenomProf <> 'BERNARD')
    AND nbCode IN (
        SELECT nbCode
        FROM T
        WHERE
            nomProf = 'FAURE'
            AND prenomProf = 'BERNARD'
    );

```

V2.

WITH

```
T (numProf, nomProf, prenomProf, code) AS (
    SELECT E.numProf, nomProf, prenomProf, code
    FROM Enseigne E
    INNER JOIN Professeur P ON E.numProf = P.numProf
)
SELECT numProf, nomProf, prenomProf
FROM T
WHERE
    nomProf <> 'FAURE'
    OR prenomProf <> 'BERNARD'
GROUP BY numProf, nomProf, prenomProf
HAVING COUNT(DISTINCT code) = (
    SELECT COUNT(DISTINCT code)
    FROM T
    WHERE
        nomProf = 'FAURE'
        AND prenomProf = 'BERNARD'
);
```

Q5 Donnez le nombre moyen d'étudiants des groupes de seconde année. *1 attribut, 1 tuple (9)*

V1.

```
SELECT AVG(nbEtudiant)
FROM (
    SELECT groupeEt, COUNT(*) AS nbEtudiant
    FROM Etudiant
    WHERE anneeEt = 2
    GROUP BY groupeEt
);
```

V2.

```
SELECT COUNT(numEt) / COUNT(DISTINCT groupeEt)
FROM Etudiant
WHERE anneeEt = 2;
```

Q6 Quel est le plus grand nombre d'étudiants à qui un professeur enseigne? *1 attribut, 1 tuple (45)*

```
SELECT MAX(nbEtudiant)
FROM (
    SELECT numProf, COUNT(DISTINCT numEt) AS nbEtudiant
    FROM Enseigne
    GROUP BY numProf
);
```

Q7 Donnez les codes et libellés des modules, et lorsqu'il existe, le pourcentage de la somme d'heures de cours par rapport au total des heures de cours dans la discipline correspondante. *3 attributs, 9 tuples*

```

SELECT code, libelle, heureCMPPrev / total AS pourcentageDiscipline
FROM MODULE
  INNER JOIN (
    SELECT SUM(heureCMPPrev) AS total, discipline
    FROM MODULE
    GROUP BY discipline
  ) NbH
  ON MODULE.discipline = NbH.discipline
WHERE
  heureCMPPrev IS NOT NULL
  AND total IS NOT NULL;

```

### 4.3 Expression des recherches récursives

**Q8** Donnez, pour chaque code de module, le nombre de professeurs qui les enseignent et la moyenne de test maximale. *3 attributs, 6 tuples*

**V1.**

```

SELECT E.code, NB_Prof, moy_max
FROM (
  SELECT code, COUNT(DISTINCT numProf) AS NB_Prof
  FROM Enseigne
  GROUP BY code
) E
  INNER JOIN (
    SELECT code, MAX(moyTest) AS moy_max
    FROM Note
    GROUP BY code
  ) N
  ON E.code = N.code;

```

**V2.**

```

SELECT E.code, COUNT(DISTINCT numProf) AS nbProfs, MAX(moyTest) AS maxMoyTest
FROM Enseigne E
  INNER JOIN Note N ON E.code = N.code
GROUP BY E.code;

```

**Q9** Quels sont les codes et libellés des racines des hiérarchies des modules? *2 attributs, 1 tuple*

```

SELECT code, libelle
FROM MODULE
WHERE codePere IS NULL;

```

**Q10** Présenter, de manière hiérarchique, les libellés de tous les sous-modules du module Informatique 2de année. *1 attribut, 16 tuples*

```

SELECT LPAD('-', 2 * LEVEL, '-') || libelle
FROM MODULE
WHERE libelle <> 'INFORMATIQUE 2DE ANNEE'
START WITH libelle = 'INFORMATIQUE 2DE ANNEE'
CONNECT BY codePere = PRIOR code;

```



**Version CTE récursive.**

```
WITH RECURSIVE
  DescModule (code, libelle, lvl) AS (
    SELECT code, libelle, 1
    FROM MODULE
    WHERE libelle = 'INFORMATIQUE 2DE ANNEE'
    UNION ALL
    SELECT M.code, M.libelle, D.lvl + 1
    FROM MODULE M
    INNER JOIN DescModule D ON M.codePere = D.code
  )
SELECT LPAD('-', 2 * lvl, '-') || libelle
FROM DescModule
WHERE libelle <> 'INFORMATIQUE 2DE ANNEE';
```

**Q11** Présenter, de manière hiérarchique, les libellés du module Outils modèles génie logiciel et de tous ses sous-modules. *1 attribut, 7 tuples*

```
SELECT LPAD('-', 2 * LEVEL, '-') || libelle
FROM MODULE
START WITH libelle = 'OUTILS MODELES GENIE LOGICIEL'
CONNECT BY codePere = PRIOR code;
```

**Version CTE récursive.**

```
WITH RECURSIVE
  DescModule (code, libelle, lvl) AS (
    SELECT code, libelle, 1
    FROM MODULE
    WHERE libelle = 'OUTILS MODELES GENIE LOGICIEL'
    UNION ALL
    SELECT M.code, M.libelle, D.lvl + 1
    FROM MODULE M
    INNER JOIN DescModule D ON M.codePere = D.code
  )
SELECT LPAD('-', 2 * lvl, '-') || libelle
FROM DescModule;
```

**Q12** Présenter, de manière hiérarchique, les libellés de tous les modules d'où est issu le module Bases de données. *1 attribut, 5 tuples*

```
SELECT LPAD('-', 2 * (MAX(LEVEL) OVER () + 1 - LEVEL), '-') || libelle
FROM MODULE
START WITH libelle = 'BASES DE DONNEES'
CONNECT BY code = PRIOR codePere
ORDER BY LEVEL DESC;
```

**Version CTE récursive.**

```

WITH RECURSIVE
  AncModule (code, libelle, codePere, lvl) AS (
    SELECT code, libelle, codePere, 1
    FROM MODULE
    WHERE libelle = 'BASES DE DONNEES'
    UNION ALL
    SELECT M.code, M.libelle, M.codePere, A.lvl + 1
    FROM MODULE M
    INNER JOIN AncModule A ON M.code = A.codePere
  )
SELECT LPAD('-', 2 * (MAX(lvl) OVER () + 1 - lvl), '-') || libelle
FROM AncModule
ORDER BY lvl DESC;

```

**Q13** Présenter, de manière hiérarchique, les libellés de tous les sous-modules de la discipline Informatique, subordonnés au module Informatique 2e année, à l'exception du module Bases de données. *1 attribut, 13 tuples*

```

SELECT LPAD('-', 2 * LEVEL, '-') || libelle
FROM MODULE
WHERE
  discipline = 'INFORMATIQUE'
  AND libelle NOT IN ('INFORMATIQUE 2DE ANNEE', 'BASES DE DONNEES')
START WITH libelle = 'INFORMATIQUE 2DE ANNEE'
CONNECT BY codePere = PRIOR code;

```

Version CTE récursive.

```

WITH RECURSIVE
  DescModule (code, libelle, discipline, lvl) AS (
    SELECT code, libelle, discipline, 1
    FROM MODULE
    WHERE libelle = 'INFORMATIQUE 2DE ANNEE'
    UNION ALL
    SELECT M.code, M.libelle, M.discipline, D.lvl + 1
    FROM MODULE M
    INNER JOIN DescModule D ON M.codePere = D.code
  )
SELECT LPAD('-', 2 * lvl, '-') || libelle
FROM DescModule
WHERE
  discipline = 'INFORMATIQUE'
  AND libelle NOT IN ('INFORMATIQUE 2DE ANNEE', 'BASES DE DONNEES');

```

#### 4.4 Expression des divisions

Quand cela possible, formulez les requêtes suivantes de deux manières différentes.

**Q14** Quels sont les noms et prénoms des professeurs ayant enseigné à tous les étudiants de seconde année? *2 attributs, 1 tuple*

Version double NOT EXISTS.

```

SELECT nomProf, prenomProf
FROM Professeur P
WHERE NOT EXISTS (
    SELECT *
    FROM Etudiant Et
    WHERE
        anneeEt = 2
        AND NOT EXISTS (
            SELECT *
            FROM Enseigne E
            WHERE
                P.numProf = E.numProf -- noqa: RF03
                AND Et.numEt = E.numEt
        )
);

```

#### Remarques

Cette requête correspond au paraphrasage “Quels sont les noms et prénoms des Professeurs tels qu’il n’existe aucun étudiant de seconde année auquel ces Professeurs n’aient pas enseigné?”

#### Version HAVING.

```

SELECT nomProf, prenomProf
FROM Professeur P
    INNER JOIN Enseigne E ON P.numProf = E.numProf
    INNER JOIN Etudiant Et ON E.numEt = Et.numEt
WHERE anneeEt = 2
GROUP BY nomProf, prenomProf
HAVING COUNT(DISTINCT Et.numEt) = (
    SELECT COUNT(*)
    FROM Etudiant
    WHERE anneeEt = 2
);

```

#### Remarques

Cette requête correspond au paraphrasage “Quels sont les noms et prénoms des Professeurs qui ont enseigné à autant d’étudiants de seconde année qu’il en existe?”

**Q15** Quels sont les noms et prénoms des étudiants dont tous les professeurs ont aussi donné cours à l’étudiant numéro 1102? *2 attributs, 11 tuples*

#### Version double NOT EXISTS.

```

SELECT nomEt, prenomEt
FROM Etudiant Et
WHERE NOT EXISTS (
    SELECT *
    FROM Enseigne E
    WHERE

```

```

numEt = 1102
AND NOT EXISTS (
    SELECT *
    FROM Enseigne E2
    WHERE
        Et.numEt = E2.numEt -- noqa: RF03
        AND E.numProf = E2.numProf
)
);

```

**Remarques**

Cette requête correspond au paraphrasage “Quels sont les noms et prénoms des étudiants tels qu’il n’existe aucun Professeur ayant eu l’étudiant numéro 1102 qui ne les ait pas eu aussi ?”

**Remarques**

Il est impossible de répondre à cette requête avec la version HAVING car il ne suffit pas que les étudiants aient eu le même nombre de Professeurs que l’étudiant numéro 1102, il faut que ces Professeurs soient les mêmes.

**Q16** Quels sont les numéros, noms et prénoms des étudiants ayant eu, pour le module Conception de SI, tous les Professeurs enseignant ce module ? *3 attributs, 3 tuples*

**Version double NOT EXISTS.**

```

SELECT numEt, nomEt, prenomEt
FROM Etudiant Et
WHERE NOT EXISTS (
    SELECT *
    FROM Enseigne E
        INNER JOIN MODULE M ON E.code = M.code
    WHERE
        libelle = 'CONCEPTION DE SI'
        AND NOT EXISTS (
            SELECT *
            FROM Enseigne E2
            WHERE
                Et.numEt = E2.numEt -- noqa: RF03
                AND E.code = E2.code
                AND E.numProf = E2.numProf
        )
)
);

```

**Remarques**

Cette requête correspond au paraphrasage “Quels sont les numéros, étudiants tels qu’il n’existe aucun Professeur enseignant le module Conception de SI qui ne leur ait pas enseigné ce module ?”

**Version HAVING.**

```

WITH
  T (numProf, numEt, libelle) AS (
    SELECT numProf, numEt, libelle
    FROM Enseigne E
    INNER JOIN MODULE M ON E.code = M.code
  )
SELECT Et.numEt, nomEt, prenomEt
FROM T
  INNER JOIN Etudiant Et ON T.numEt = Et.numEt
WHERE libelle = 'CONCEPTION DE SI'
GROUP BY Et.numEt, nomEt, prenomEt
HAVING COUNT(DISTINCT numProf) = (
  SELECT COUNT(DISTINCT numProf)
  FROM T
  WHERE libelle = 'CONCEPTION DE SI'
);

```

#### Remarques

Cette requête correspond au paraphrasage “Quels sont les numéros, noms et prénoms des étudiants dont le nombre de Professeurs en Conception de SI est égal au nombre total de Professeurs enseignant ce module?”

**Q17** Quels sont les codes et libellés des modules suivis par tous les étudiants du groupe d’étudiants 2 de seconde année? *2 attributs, 1 tuple*

**Version double NOT EXISTS.**

```

SELECT code, libelle
FROM MODULE M
WHERE NOT EXISTS (
  SELECT *
  FROM Etudiant Et
  WHERE
    anneeEt = 2
    AND groupeEt = 2
    AND NOT EXISTS (
      SELECT *
      FROM Enseigne E
      WHERE
        M.code = E.code -- noqa: RF03
        AND Et.numEt = E.numEt
    )
);

```

#### Remarques

Cette requête correspond au paraphrasage “Quelles sont les modules tels qu’il n’existe aucun étudiant du groupeEt 2 de seconde année qui ne les suit pas?”

**Version HAVING.**

```

SELECT M.code, libelle
FROM Etudiant Et
    INNER JOIN Enseigne E ON Et.numEt = E.numEt
    INNER JOIN MODULE M ON E.code = M.code
WHERE
    anneeEt = 2
    AND groupeEt = 2
GROUP BY M.code, libelle
HAVING COUNT(DISTINCT Et.numEt) = (
    SELECT COUNT(numEt)
    FROM Etudiant
    WHERE
        anneeEt = 2
        AND groupeEt = 2
);

```

#### Remarques

Cette requête correspond au paraphrasage “Quelles sont les modules suivis par autant d’étudiants du groupeEt 2 de seconde année qu’il en existe?”

## 4.5 Requêtes complexes

**Q18** Présenter, de manière hiérarchique, les libellés de tous les sous-modules de la discipline Informatique subordonnés au module Informatique 2de année à l’exception du module Codage et circuits et de tous ses éventuels sous-modules. *1 attribut, 13 tuples*

```

SELECT LPAD('-', 2 * LEVEL, '-') || libelle
FROM MODULE
WHERE discipline = 'INFORMATIQUE'
START WITH libelle = 'INFORMATIQUE 2DE ANNEE'
CONNECT BY codePere = PRIOR code
AND libelle <> 'CODAGE ET CIRCUITS';

```

Version CTE récursive.

```

WITH RECURSIVE
    DescModule (code, libelle, discipline, lvl) AS (
        SELECT code, libelle, discipline, 1
        FROM MODULE
        WHERE libelle = 'INFORMATIQUE 2DE ANNEE'
        UNION ALL
        SELECT M.code, M.libelle, M.discipline, D.lvl + 1
        FROM MODULE M
            INNER JOIN DescModule D ON M.codePere = D.code
        WHERE M.libelle <> 'CODAGE ET CIRCUITS'
    )
SELECT LPAD('-', 2 * lvl, '-') || libelle
FROM DescModule
WHERE discipline = 'INFORMATIQUE';

```

**Q19** Présenter, de manière hiérarchique, les modules suivis par l’étudiant Jérôme Atlani? *1 attribut, 24 tuples*

```

SELECT LPAD('-', 2 * (MAX(LEVEL) OVER () + 1 - LEVEL), '-') || libelle
FROM MODULE
START WITH code IN (
    SELECT code
    FROM Enseigne E
    INNER JOIN Etudiant Et ON E.numEt = Et.numEt
    WHERE
        nomEt = 'ATLANI'
        AND prenomEt = 'JEROME'
)
CONNECT BY code = PRIOR codePere;

```

Version CTE récursive.

```

WITH RECURSIVE
AncModule (code, libelle, codePere, lvl) AS (
    SELECT M.code, M.libelle, M.codePere, 1
    FROM MODULE M
    WHERE
        M.code IN (
            SELECT E.code
            FROM Enseigne E
            INNER JOIN Etudiant Et ON E.numEt = Et.numEt
            WHERE
                nomEt = 'ATLANI'
                AND prenomEt = 'JEROME'
        )
    UNION ALL
    SELECT M.code, M.libelle, M.codePere, A.lvl + 1
    FROM MODULE M
    INNER JOIN AncModule A ON M.code = A.codePere
)
SELECT LPAD('-', 2 * (MAX(lvl) OVER () + 1 - lvl), '-') || libelle
FROM AncModule;

```

#### Remarques

Il n'existe pas de manière simple pour présenter, de manière hiérarchique, chacune des arborescences. L'utilisation de la clause ORDER BY aurait pour effet de les mélanger, pas de les trier comme souhaité.

**Q20** Quels sont les numéros, noms et prénoms des étudiants de seconde année dont toutes les moyennes de test sont supérieures ou égales à 10 ? *3 attributs, 23 tuples*

V1.

```

SELECT Et.numEt, nomEt, prenomEt
FROM Note N
    INNER JOIN Etudiant Et ON N.numEt = Et.numEt
WHERE
    anneeEt = 2
    AND moyTest >= 10

```

```
GROUP BY Et.numEt, nomEt, prenomEt
HAVING COUNT(moyTest) = (
    SELECT COUNT(moyTest) -- noqa: RF03
    FROM Note N2
    WHERE Et.numEt = N2.numEt -- noqa: RF03
);
```

V2.

```
SELECT Et.numEt, nomEt, prenomEt
FROM Note N
    INNER JOIN Etudiant Et ON N.numEt = Et.numEt
WHERE anneeEt = 2
GROUP BY Et.numEt, nomEt, prenomEt
HAVING MIN(moyTest) >= 10;
```

**Q21** Quels sont les numéros, noms et prénoms des étudiants ayant des moyennes dans un module subordonnés au module Principes des BD? *3 attributs, 19 tuples*

```
SELECT DISTINCT Et.numEt, nomEt, prenomEt
FROM Etudiant Et
    INNER JOIN Note N ON Et.numEt = N.numEt
WHERE code IN (
    SELECT code
    FROM MODULE
    START WITH libelle = 'PRINCIPES DES BD'
    CONNECT BY codePere = PRIOR code
);
```

Version CTE récursive.

```
WITH RECURSIVE
    DescModule (code) AS (
        SELECT code
        FROM MODULE
        WHERE libelle = 'PRINCIPES DES BD'
        UNION ALL
        SELECT M.code
        FROM MODULE M
            INNER JOIN DescModule D ON M.codePere = D.code
    )
SELECT DISTINCT Et.numEt, nomEt, prenomEt
FROM Etudiant Et
    INNER JOIN Note N ON Et.numEt = N.numEt
WHERE code IN (SELECT code FROM DescModule);
```

**Q22** Donnez les numéros des étudiants, les écarts entre leurs éventuelles moyennes de test et la moins bonne moyenne de test du module ACSI ainsi que les écarts entre leurs éventuelles moyennes de test et la meilleure moyenne de test du module ACSI. *3 attributs, 29 tuples*

V1.



```
WITH
  T1 (moyMin, moyMax) AS (
    SELECT MIN(moyTest), MAX(moyTest)
    FROM Note
    WHERE code = 'ACSI'
  )
  , T2 (numEt, moyTest) AS (
    SELECT numEt, moyTest
    FROM Note
    WHERE code = 'ACSI'
  )
SELECT numEt, moyTest - moyMin AS ecartMin, moyTest - moyMax AS ecartMax
FROM T1, T2;
```

V2.

```
SELECT numEt, moyTest - moyMin AS ecartMin, moyTest - moyMax AS ecartMax
FROM Note, (
  SELECT MIN(moyTest) AS moyMin, MAX(moyTest) AS moyMax
  FROM Note
  WHERE code = 'ACSI'
) T
WHERE code = 'ACSI';
```

V3.

```
SELECT
  numEt
  , moyTest - (
    SELECT MIN(moyTest)
    FROM Note
    WHERE code = 'ACSI'
  ) AS moyMin
  , moyTest - (
    SELECT MAX(moyTest)
    FROM Note
    WHERE code = 'ACSI'
  ) AS moyMax
FROM Note
WHERE code = 'ACSI';
```